

Computer Graphics

Instructor: MSc. Thai Trung Tin

Email: ttin@hcmiu.edu.vn

Part 1: In-Class Practices and Exercises

Duration: 3 hours

Material:  Lab 2

CG Lab 2: Transformation

In the previous lab homework, we studied the Shader, PyGame, and PyOpenGL. In this Lab, we will study the uniform data and transformation.

Exercise 1: UNIFORM DATA (chapter 3 in the textbook)

Based on *animation-1.py* and *animation-2.py* files, about the animation of a triangle sliding horizontally across the screen and a triangle orbiting in a circular path around the origin.

Part A – Vertex Transformation

1. The vertex shader applies:
$$gl_Position = position + translation$$
 - Write down the general form of this transformation in matrix form (homogeneous coordinates).
 - Explain why we need the extra '1' in the homogeneous coordinate system.

Part B – Translation in Animation-1

1. The update rule is:
$$x' = x + \Delta x \quad \text{with } \Delta x = 0.03 \text{ per frame}$$
 - Write the translation matrix for a 2D translation by (t_x, t_y) :

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- Apply this to one vertex of the triangle, e.g., (0.0, 0.2, 1), and show the result after a few frames.
2. Why does the program reset the x-translation when $x > 1.2$?
- Relate this to the clip space range in OpenGL.

Part C – Circular Motion in Animation-2

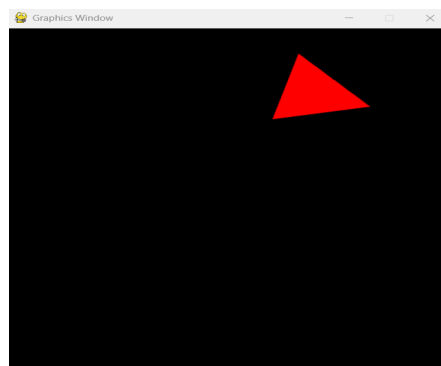
1. The update rule is:

$$x = r * \cos(\theta), \quad y = r * \sin(\theta)$$
 with $r = 0.75, \theta = \text{time}$
 - Derive these equations from the unit circle definition.
 - What is the translation matrix for a point (0,0,1) moved to (x,y) on the circle?
2. Show how the vertex (0.0, 0.2) is transformed at:
 - $t = 0$
 - $t = \pi/2$
 - $t = \pi$

Part D - Implementation

1. Orbiting & Spinning Triangle

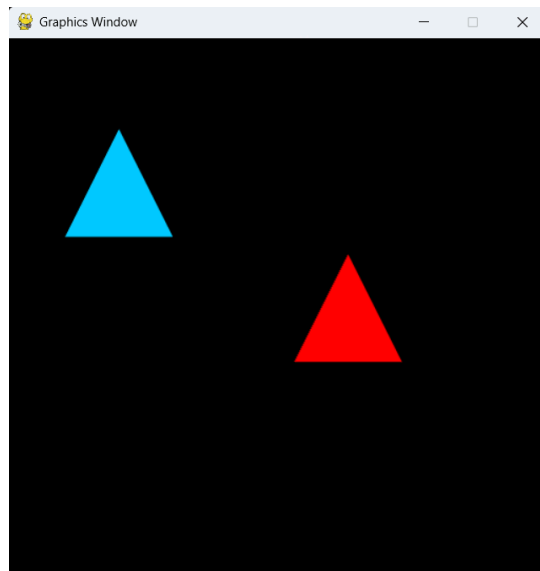
Implement a program that draws a triangle that **orbits around the origin** and **spins around its own center** at the same time.



- Use trigonometric equations $x = r \cdot \cos(\theta)$, $y = r \cdot \sin(\theta)$ for the orbit.
 - Use a rotation matrix to spin the triangle.
 - Combine both transformations inside your vertex shader.
2. Two Triangles (Linear vs Circular Motion)

Implement a program that draws **two triangles simultaneously**:

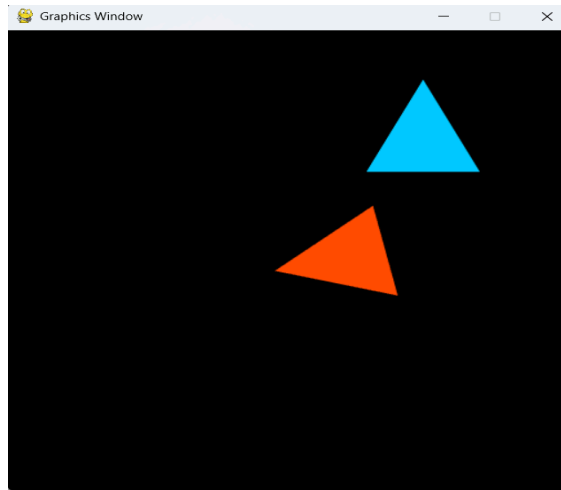
- The first triangle should move **linearly across the screen** and reset when leaving the right side.
- The second triangle should move in a **circular path** around the origin using cosine and sine.
- Both should be drawn in different colors.



3. Linear Motion with Rotation + Circular Motion

Implement a program that draws **two triangles with different behaviors**:

- Triangle A: moves **linearly across the screen** and also **rotates** around its own center while moving.
- Triangle B: moves in a **circular orbit** around the origin.
- Use a combined transformation $M = T(T_x, 0) \cdot R(\theta)$ for Triangle A.



Exercise 2: Matrix Algebra and Transformations (chapter 4 in the textbook)

You are provided 3 Python scripts:

1. `key-control.py`

This program allows the user to control a triangle's position on screen using the arrow keys.

- The triangle is drawn at an initial offset (translation = $[-0.5, 0.0, 0.0]$).
- The vertex shader adds the translation uniform to the vertex position.
- In the update loop, pressing the arrow keys ($\leftarrow \rightarrow \uparrow \downarrow$) adjusts the translation values, effectively moving the triangle across the screen.
- The movement speed is frame-rate independent because it uses `delta_time`.

2. `global-and-local-transformations.py`

This program demonstrates the difference between global and local transformations applied to a triangle.

- The triangle starts at the center and can be moved or rotated using keyboard keys.
- Global transformations (keys *WASDZXQE*) move or rotate the triangle relative to the world origin (0, 0, 0). For example, pressing W moves the triangle upward in the global coordinate system, regardless of its current orientation.
- Local transformations (keys *IJKLUO*) move or rotate the triangle relative to its own local coordinate system. For example, pressing I always moves the triangle “forward” in its own orientation.
- This highlights the importance of matrix multiplication order:
 - o Global: $M \leftarrow T \cdot M$
 - o Local: $M \leftarrow M \cdot T$

3. Spinning Sphere - sphere.py

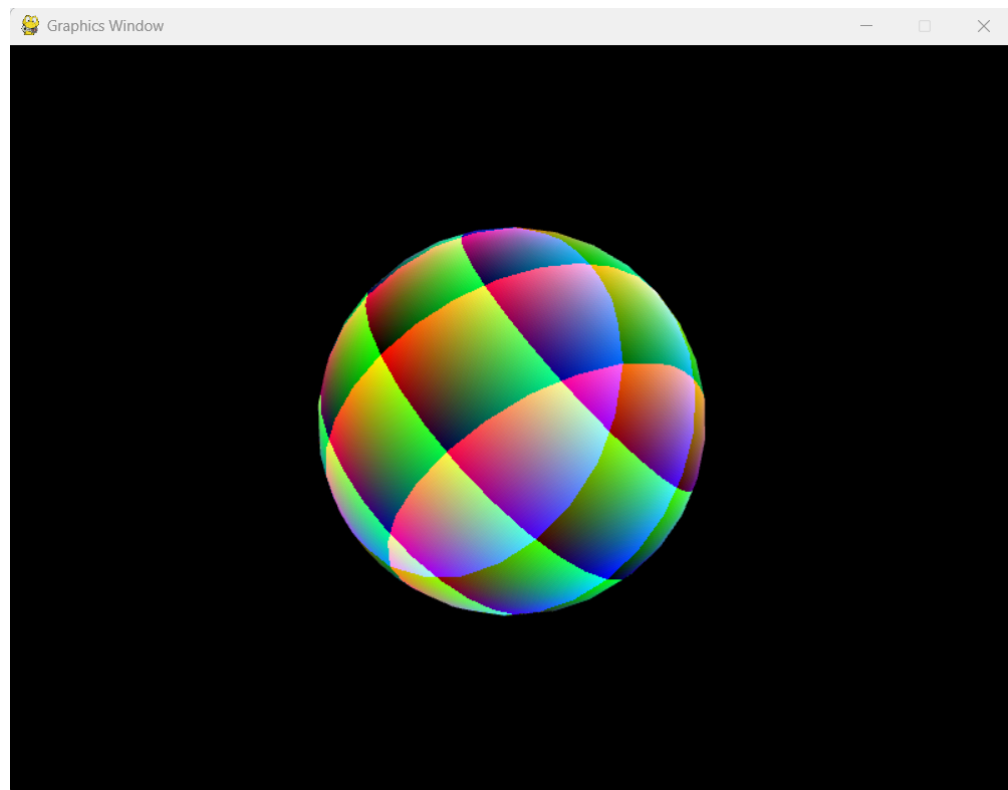
This program renders a 3D spinning sphere with gradient colors.

- Uses SphereGeometry to create the sphere mesh.
- A custom **vertex shader** applies the model, view, and projection matrices, transforming the sphere’s vertices into clip space.
- The fragment shader generates color based on the vertex position, producing a gradient effect.
- The sphere is animated by applying small rotations each frame:

```
self.mesh.rotate_y(0.00514)
self.mesh.rotate_x(0.00337)
```

- The scene uses a Camera, Renderer, and Scene setup typical of modern 3D frameworks.

In this assignment, you will extend the spinning sphere program to add interactive controls. The final result should allow the user to move the sphere using both global and local transformations, rotate the sphere interactively, and keep the sphere auto-spinning in the background.



Part A – Adding Keyboard Controls

1. Modify the sphere program so that the sphere can be moved with the keyboard:
 - Use keys WASDZX for global translations (world space).
Example: pressing W should always move the sphere upward in the world, regardless of its orientation.
 - Use keys IJKL for local translations (object space).
Example: pressing I should move the sphere “forward” relative to its current orientation.
2. Explain why global movement uses: $M = T * M$, while local movement uses: $M = M * T$.

Part B – Interactive Rotations

1. Add key controls to rotate the sphere left/right around the Y-axis.
 - Use keys Q and E for global rotation.
 - Use keys U and O for local rotation.
2. Write down the rotation matrix used:
$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
3. Explain the difference between rotating around the world axis and rotating around the object's local axis.

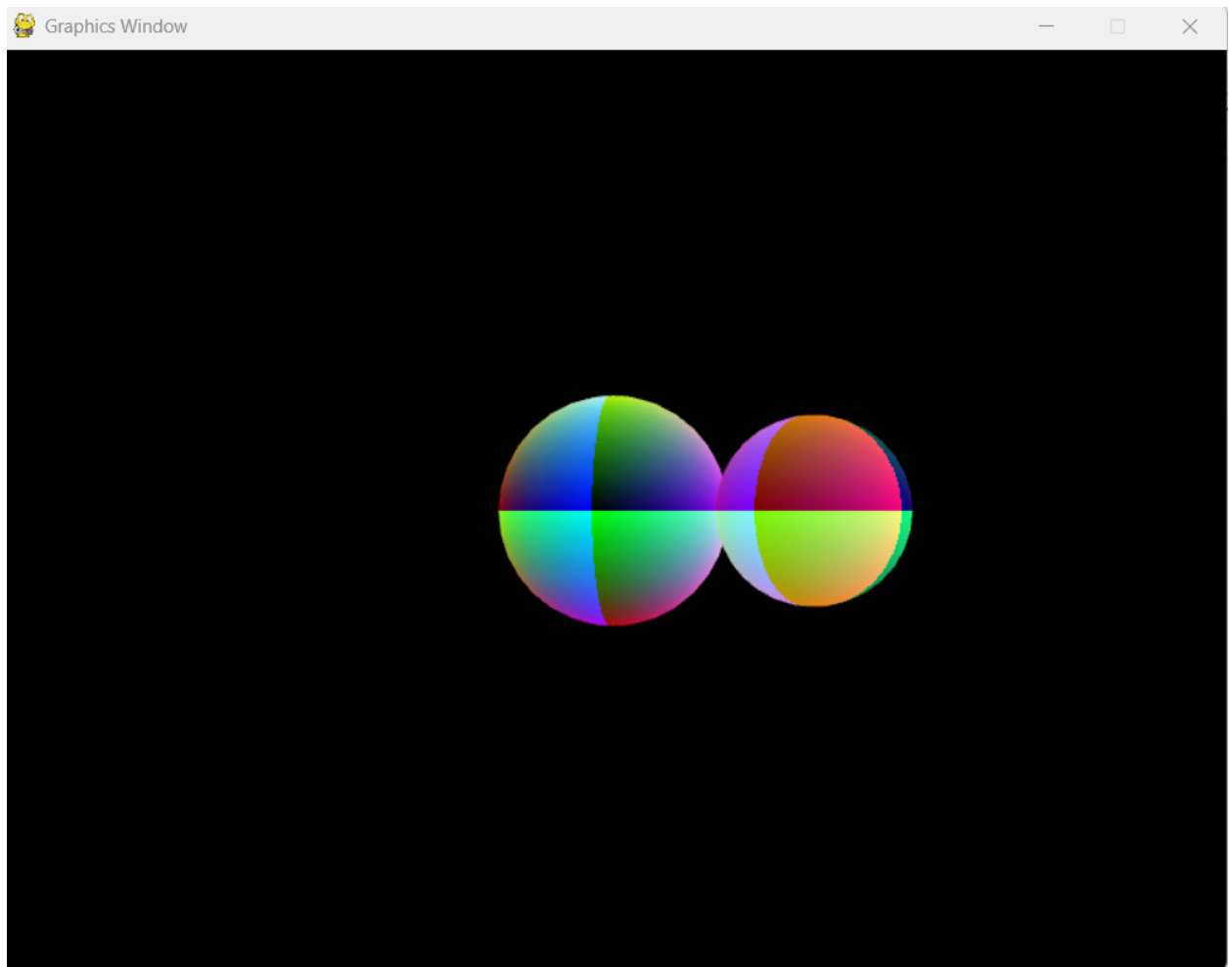
Part C – Auto-Spin and Combination

1. Keep the sphere auto-spinning around both the X and Y axes (like the original sphere program).
2. Combine auto-spin with your key controls. Verify that:
 - The sphere spins automatically.
 - You can still move and rotate it interactively using both global and local keys.
3. Write the full transformation pipeline equation for a vertex in your program:
$$gl_Position = Projection \times View \times Model \times Position$$

Part 2 Home work: Extending the Spinning Sphere

You have already seen how to draw a single spinning sphere with gradient colors. In this homework, you will extend that idea to simulate a simple **Solar System** with two spheres:

- A **Sun** at the center that spins on its axis.
- An **Earth** that both spins on its axis and moves in a circular orbit around the Sun.



Question 1 – Spinning Sun

Start with the given `sphere.py` example.

1. Modify the program so that the **Sun (radius = 1.0)** spins continuously around its Y-axis.
2. Use the gradient shader (`mod(position, 1.0)`) so the spinning motion is clearly visible.
3. Show the code where you added the rotation.

Question 2 – Adding the Earth (Orbit Only)

1. Add a second sphere (radius = 0.5) to represent the Earth.
2. Place the Earth so that it **orbits the Sun** at a radius of 3.0 units.
Use cosine and sine functions:
 $x=R \cdot \cos(\theta)$, $z=R \cdot \sin(\theta)$
3. Make sure the Earth also uses the same gradient color shader.

Run the program and describe what you see.

Question 3 – Earth's Self-Spin

1. Extend the Earth so that it also **spins on its own Y-axis** while orbiting.
2. Write the transformation equation for Earth's model matrix, combining:
 - Orbit translation
 - Self rotation
3. Explain why both transformations are needed to simulate realistic Earth motion.

Question 4 – Adding the Moon

1. Add a third sphere (radius = 0.2) to represent the Moon.
2. The Moon should orbit around the **Earth** at a smaller radius (e.g., 1.0).
 - Use cosine and sine to update its position relative to Earth:
$$x_{\text{moon}} = x_{\text{earth}} + R \cdot \cos(\phi), \quad z_{\text{moon}} = z_{\text{earth}} + R \cdot \sin(\phi)$$
 - Where $(x_{\text{earth}}, z_{\text{earth}})$ is Earth's position from Question 2.
3. Make the Moon also **spin on its own Y-axis**.

Run the program — you should now see a spinning Sun, a spinning Earth orbiting the Sun, and a spinning Moon orbiting the Earth.

